Linear Layouts From First Principles

Thursday, January 15, 2026

This post explains what linear layouts are and why they're cool.

There is a paper if you prefer, and also two blog posts, by Lei Zhang from AMD. The source code in Triton may also be illuminating.

I want to use this post to explain the concept more informally, and to build intuition for why you'd want them in the first place.

The initial idea behind linear layouts is due to Adam P. Goucher; I was just one of the people who worked on the implementation in Triton.

What is a layout?

Let's start with the definition of a *layout*.

In ML languages like JAX and Triton, users write their code in terms of *tensors*, which are just multi-dimensional arrays of numbers. For example, you might have a 2D matrix composed of rows and columns; or a 4D image with dimensions batch, height, width, and channels.

Although the user thinks of their tensor as an N-dimensional array, at some point the elements probably need to be stored in memory. Memory is usually thought of as a one-dimensional array, so to store the tensor, we somehow need to flatten an N-dimensional index (e.g. the tuple `(row, column)`) into a one-dimensional index.

Informally, a function that maps an N-dimensional index to a 1D index is called a *layout*. For example, two common layouts for a 2D array are row-major and column-major.

GPUs can become very slow if your reads and writes don't use "good" memory access patterns. Therefore the choice of layout is often critical to performance.

In principle, if you have a tensor with n elements, there are $n!$ possible layouts. But in practice we don't want our layouts to be arbitrary functions. Just writing down an arbitrary layout for a tensor with 1B elements would require at minimum

$\log_2(10^9!)$ bits, approximately 3.6GB! So the layout would be roughly as big as the tensor itself.

So the trick is coming up with a class of layout functions that are

- powerful enough to be useful, and
- constrained enough to be practical.

Linear layouts are one such class of functions. As we'll see, they're particularly useful for GPUs.

Before I show you what linear layouts are, I want to motivate the design. And to do that, I need to define what "useful" is. Let's go through some examples of layouts we want to be able to represent.

Examples of "useful" layouts

Layout #1: Permutation of the input dimensions

The simplest layout is probably a permutation of the dimensions of the tensor.

For example, a 2D array of size $R \times C$ can be stored in row-major or column-major order. We can write these as functions as follows.

$$\begin{aligned}\text{RowMaj}(r, c) &= C \times r + c \\ \text{ColMaj}(r, c) &= R \times c + r\end{aligned}$$

If we want to represent a layout of this form, we just need to write down a permutation of the input dimensions. Then we flatten them into a 1D index using that ordering (e.g. "rows then columns" or "columns then rows").

Layout #2: Reorder the input bits

Permuting the dimensions covers 90% of interesting layouts; XLA:GPU got away with this for a long time. But eventually you'll probably want to represent a "tiled layout".

For example, nvidia's cuDNN has the CUDNN_TENSOR_NCHW_VECT_C layout for 4D image tensors with dimensions batch (N), channel (C), height (H), width (W).

Here's one way to understand this layout. Assume the dimensions of the tensor are all powers of 2. Given an input index (n, c, h, w), construct a 1D output index by concatenating bits from the input indices as follows (starting with the high-order bits):

- All the bits of n.

- All but the two least-significant bits of c.
- All the bits of h.
- All the bits of w.
- The two least-significant bits of c.

You can see this is kind of a compromise between `NCHW` (i.e. `N` is most major, `W` is most minor) and `NHWC` (i.e. `C` is most minor); we've effectively split the `C` dimension into two sub-dimensions. Splitting up a dimension like this is what we mean by "tiling". In the case of `VECT_C`, tiling lets us write more efficient kernels for tensors that have small elements, such as `int8`.

We can generalize this into a layout that allows us to reorder the bits of the input index arbitrarily:

- Concatenate all the input dimensions into a single bitstring b.
- Reorder the bits of b according to a permutation P.
- Return this value as the layout's 1D index.

To represent such a layout, we only need to write down the permutation P, which is small (log of the number of elements in the tensor).

You can see that this kind of layout subsumes the case where we permute the dimensions, plus we can now represent tiled layouts. Great!

Notice that with this scheme, having multiple input dimensions doesn't give us any additional power; we just concatenate them into a single index b. Multiple input dimensions are still useful to the user, but it's a nice property that we can specify the same effective layout no matter how the user decides to split up their logical dimensions.

Indeed, another way of thinking of this is that it's the same as the earlier case where we permute the dimensions, except we reshape the input so that all dimensions have size 2. Then we have $\log n$ input indices, and our permutation P specifies how to permute them to get the output index.

A downside of this scheme as compared to the first one is, we lost the ability to represent non-power-of-2 dimension sizes. We also lost the ability to represent layouts for tensors where we don't know the size of the dimensions ahead of time (sometimes called "dynamic shapes"). In practice we can usually handle this by choosing a fixed-sized inner "block". We can then represent the tensor as a dynamically-sized array of blocks and use linear layouts to represent the contents of one block.

The next problem you might run into has to do with avoiding shared memory bank conflicts.

nvidia GPUs have a special memory space called "shared memory". For our purposes, the important thing about it is that it has 32 "banks". In one cycle, you can read one element from each bank. But if you try to read n elements from the same bank, that's called a "bank conflict", and it takes n cycles. It's therefore important to arrange your data so that your reads and writes avoid bank conflicts.

Suppose we want to store a 32x32 matrix in shared memory, and suppose the accesses we want to support are:

1. Read all the elements of one row.
2. Read all the elements of one column.

Suppose we naively were to store the matrix in shared memory in row-major order. I claim this will have lots of bank conflicts. Let's see why.

Our naive layout function is:

$$\text{ShmemRowMaj}(r, c) = (\text{row} = r, \text{bank} = c)$$

(Up until now our output indices were a single number. Here it's conceptually cleaner to split it into two numbers, but we can always combine them back into a single number as $32 \times \text{row} + \text{bank}$.)

Now we analyze the memory access patterns to check for bank conflicts.

- When we read one row r, we read elements $(r, 0), (r, 1), \ldots (r, 31)$. These are in banks $0, 1, \ldots, 31$. Great, no bank conflicts!
- But now suppose we try to read one column c. We read elements $(0, c), (1, c), \ldots, (31, c)$. These are all in the same bank, namely c. So that's bad.

OK, so row-major doesn't work. You can convince yourself that column-major has the same problem. Here's a layout that *does* work.

$$\text{ShmemSwizzled}(r, c) = (\text{row} = r, \text{bank} = r \oplus c)$$

where $\oplus$ represents bitwise xor.

Let's check that this avoids bank conflicts for our desired memory access pattern.

- When we read one row r, we read elements $(r, r \oplus 0), (r, r \oplus 1), \ldots, (r, r \oplus 31)$. The banks are $r \oplus 0, r \oplus 1, \ldots, r \oplus 31$. To see that there are no duplicates, observe that we could write the list as $r \oplus (0, 1, \ldots, 31)$. Xor is an invertible operation, so because the elements $0, 1, \ldots, 31$ are all different, the elements are all different after xor'ing with r.

- When we read one column c, we read elements $(0, 0 \oplus c), (1, 1 \oplus c), \ldots, (31, 31 \oplus c)$. The banks accessed can be written as $(0, 1, \ldots 31) \oplus c$, and by the same argument as above, all of these are different.

This might seem like a weird way to achieve our goal of avoiding bank conflicts on row/col accesses, and there are in fact other more intuitive ways to accomplish the same thing. But one nice property of this approach is that it's very fast to compute.

We can generalize this to allow arbitrary swizzling, as follows.

- Concatenate all the input dimensions into a single bitstring b.

- For each i, the layout defines a bitmask m_i of the same length as b. (The masks are a property of the layout, independent of the value of b.)

- The i-th output bit o_i is the parity (i.e. xor-sum) of the subset of the bits of b specified by m_i. Put another way, let o_i equal 1 if $b \otimes m_i$ has an odd number of 1s (where $\otimes$ is bitwise and).

A layout specified by a list of masks m_i in this way is what we call a *linear layout*. Specifying the list of masks is tractable because the number and size of the bitmasks is logarithmic in the number of elements in the tensor.

Our shared memory example above can be written in this form. For the first dimension of the output, our mask simply selects the i-th bit of the input r. For the second dimension of the output, our mask selects the i-th bit of the input r and the i-th bit of the input c.

Critically, this scheme can also represent the layouts that nvidia and AMD GPUs use for tensor core operations.

You can check that this scheme subsumes the previous one where we reorder the bits of the input index. In this case each m_i has a single 1 bit, corresponding to the input bit that goes to output bit i.

What's cool about this is that we can represent a bunch of different layout transformations using the same machinery. Transposes, reshapes, tiling, and multiple generations of nvidia and AMD tensor core layouts are all just special cases of linear layouts. Moreover, as we'll see, we can *combine* any of these transformations. This is all much nicer than having special-purpose code for e.g. swizzled layouts vs tensor-core layouts.

But the rule for applying a linear layout might seem kind of magic. Where does it come from, and what's "linear" about it? Let's look at that next.

A Mathematical Perspective on Linear Layouts

Recall that given a layout's masks m_i and an input index b (formed by concatenating all of the input dimensions), the i-th bit of the output index o_i is computed as follows.

$$o_i = \bigoplus_j (m_i \otimes b)_j$$

where $\oplus$ is bitwise xor, and $\otimes$ is bitwise and. The "sum" is over the bits of $m_i \otimes b$. (I'm abusing notation here; the subscript j is a bit index, whereas the subscript i in m_i refers to the i-th bitmask.)

If you've done linear algebra, maybe you're onto me at this point. With the notation I've chosen, this looks a lot like the dot product of the vectors m_i and b! And indeed we can think of it that way.

First we need to define the field we're working on. Let $\mathbb{F}_2$ be the field with two elements, 0 and 1. Let addition on this field be `xor`, and multiplication be `and`.

Now we can interpret b, m_i, and o as vectors in $\mathbb{F}_2^n$. Then indeed $o_i = m_i \cdot b$.

Moreover we notice that we're doing repeated dot products here, so that's really a matrix-vector multiplication! We can stack the m_i vectors as the rows of a matrix M and then say that $o = Mb$.

In other words, a linear layout function is just a linear transformation over the field $\mathbb{F}_2^n$! That's where "linear" in "linear layout" comes from.

As a worked example, suppose our input index is a 3-bit vector $b = (b_2, b_1, b_0)$.

Define a layout using the following masks:

$$m_0 = (1, 0, 1), \qquad m_1 = (0, 1, 1), \qquad m_2 = (1, 1, 1).$$

Then the output bits are

$$o_0 = b_2 \oplus b_0, \qquad o_1 = b_1 \oplus b_0, \qquad o_2 = b_2 \oplus b_1 \oplus b_0.$$

In matrix form, this is

$$\begin{pmatrix} o_0 \\ o_1 \\ o_2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} b_2 \\ b_1 \\ b_0 \end{pmatrix},$$

where all arithmetic is over $\mathbb{F}_2$.

A reasonable question you might have is, is *any* matrix a valid linear layout? And it's easy to see, no. For example, the zero matrix would map every input index to memory location 0, but it doesn't make sense to have multiple elements stored in the same memory location. Therefore we require our transformation to be injective, i.e. to map each input to a different output.

We don't necessarily need to require surjectivity, i.e. we *can* allow some memory locations to have no corresponding input index. This would be a form of padding. But we already require padding the input size to be a power of two; usually we don't need *more* padding than that, so we usually require surjectivity.

Another question you might have is, what if I want the same tensor element to be stored in two different memory locations (i.e. we want a broadcasting layout)? As defined so far, we can't do this; it would require us to turn our transformation into a multivalued function, which in turn would prevent us from representing it as a linear transformation.

But what we can do is swap the input and output of our layout function. Instead of mapping from input indices to memory locations, we can map from memory locations to input indices. Each memory location maps to one input index, but multiple memory locations can map to the same input index. This is actually how the paper defines linear layouts. I just didn't do it this way in this writeup because it's unintuitive to think of going in this direction. If we don't have broadcasting, the two representations are equivalent; you just invert the matrix to go between them. (The matrix is invertible if and only if there is no broadcasting.)

What's cool about this is that we can now do math on our layouts. For example:

- Want to invert a layout? Just invert the matrix M!
- Want to compose two layouts? Just multiply their matrices!
- Suppose we have an input tensor with layout L_1 and an output tensor with layout L_2. We can use shared memory to shuffle the input into the output. But

what layout should we use for the shared memory to avoid bank conflicts, while still allowing us to read from the input and write to the output efficiently? We can now write an algorithm to find this! It's described in the paper under the heading "optimal swizzling".

The values in the matrix M also have a natural interpretation. If you have an input with value 2^i (i.e. a bitstring with a single 1 in the i-th position), then the output value is simply the i-th column of M. So specifying a layout's matrix is equivalent to specifying how it operates on power-of-two inputs.

Conclusion

When people talk about layouts, there are usually lots of diagrams. I like diagrams in general, but I almost always find layout diagrams hard to understand. One of the things that I love about linear layouts is, we don't need to do this. You can write a concise and precise description of a layout as a matrix, and then I can translate that directly into code. Done.

There's more to say, but I think I'll stop here. This should give you intuition if you want to read the paper or work with linear layouts inside Triton!

Thanks to Michael Kuperstein, Sanjoy Das, Haggai Nuchi, and ChatGPT 5.2 Pro for feedback on drafts of this post.

Published using Weblog